Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous

Abstract

Checking a stored proof can avoid its original search, while checking a computational certificate can repeat the calculation that justifies it. We measure these costs in a pinned, published AI-generated Lean library and use the findings to design ProofServe, an original-kernel check and audit tool. Matched target windows retain 80.28–82.35% of source-compilation wall time for two computational certificates, versus 0.46–6.54% for two ordinary controls. Across 31 module-process pairs, the median checker/compiler CPU ratio is 0.451, with overlapping source-proxy classes. A per-module campaign accounts for P-core and E-core CPU separately; its report will distinguish measured coverage from a complete library total, with the original probability sample retained for calibration. The default import graph omits 19.78% of source modules and 33.30% of lexical kernel-decision sites. ProofServe combines dependency-aware parallel replay, restartable segments and receipts recording scope, axioms and costs. Its contribution is a usable integration of established mechanisms and scoped evaluation. Final release scaling and external-project measurements remain pending.

1 Introduction

Proof-embedded computation can preserve expensive work across the boundary between producing and checking an AI-generated proof. Lean validates elaborated terms rather than repeating tactic search, but definitional equality can still evaluate a program inside a term. Users therefore need both a cost model and a precise account of what a successful project check covers.

Existing work already establishes the value of independent checking. Lean 4.35 ships `lake check`, following the FRO's external-checking agenda [8, 7]. A public report on the same OpenAI library performs a complete cloud build and extensive independent checking [10]. It proposes kernel computation as an explanation for expensive checks from three source examples. Our empirical increment is a matched module-level measurement of that explanation, a per-module census with explicit condition-dependent accounting, and a coverage analysis at a different corpus revision. Our systems contribution is ProofServe's original-kernel workflow and auditable receipts, evaluated through its release-form interface. The four questions follow these boundaries:

RQ1 How do checking costs differ between stored mathematical proofs and computational certificates?

RQ2 How is compilation work distributed across the pinned library?

RQ3 How does parallel original-kernel checking trade wall time, CPU work and additional trusted code?

RQ4 What do default builds, repeated verdicts and axiom receipts establish about verification coverage?

2 Method and workload

A per-module census and matched checking windows separate library work from selected-case checking behavior. Our primary corpus is the published OpenAI mathematics repository [11], pinned at `4eaee875`: 163,097 source modules, Lean 4.34.1 and Mathlib `d13f23b7`. Two original draws in each of 21 strata combine source-size percentile, import depth and an explicit kernel-decision marker. All 42 now have completed compilation costs. Prerequisites and selected hot examples retain their separate identities and weights.

Source cost includes a module's imports, tactics, kernel checking and artifact emission. Separate prerequisite preparation, C compilation and build-driver work are outside this clock. CPU means user plus system time. Whole-process replay and incremental target windows are distinct measurements; Figure 1 labels both. Reported sample and release timings use CPUs 0–7 of an i9-14900K, one task at a time, retaining file caches. Lean runs omit `-M`, including `-M0`; external memory protection is recorded separately. Earlier in-process memory checks and missing prototype persistence marking were our own method defects.

The public package `data/openai-math-4eaee875/` records sample identities, raw-cost provenance, clocks and estimates. This paper pins its original-sample summary to `1cc14067`. Historical replay measurements explain cost boundaries; system performance must come from committed release-tool results in `checker/results/` and `data/external-projects/`.

The authorized census uses a frozen eight-P-core/fourteen-E-core protocol. Its total-instruction invariance gate failed: median spread 1.33% and P95 3.15% exceeded the unchanged 1% and 3% criteria. Cross-module comparisons use user-mode instructions, a criterion chosen after observing those results. P-core and E-core CPU seconds are reported separately without conversion; wall time describes this machine and configuration. Full-corpus numerical results are not yet available.

3 Two costs and their population weight

3.1 Computational certificates retain work

Matched target windows show expensive replay for computational certificates and cheap replay for ordinary controls. In Figure 1(b), Grid and Amplitudes retain 80.28% and 82.35% of their source compiler's wall time. Their replay components are 255.489 and 110.026 seconds; almost all of each is inside the kernel replay call. Mathlib Nat.Basic and NearPin retain 6.54% and 0.46%. Each target component reads, maps and checks that target after dependency preparation; each denominator is that module's compiler process. These shared boundaries quantify the expensive kernel replay anticipated by the report's computation hypothesis in the selected cases [10].

The mechanism is visible in the proof boundary. A tactic can search for a mathematical derivation and emit a term that records its successful steps. Replaying that term validates those steps without reconstructing the search. A finite certificate justified through `of_decide_eq_true` instead asks the kernel to validate the decision procedure's result through reduction. The compact wrapper can therefore retain evaluation over a large finite object. Our target measurement holds the declared input and original checking operation fixed and locates that retained cost in the replay window. Its practical consequence is that source length and a cheap closure average are weak planning signals for an individual certificate.

The broader paired panel shows variation rather than a clean binary split. Across 31 original modules, checker/compiler CPU has median 0.451: 0.391 for the 14 unmarked and 0.498 for the 17 marked modules. An explicit comment/string-masked `decide +kernel` site is a reproducible proxy for computation, while ordinary `decide` can also encode substantial evaluation [2, 5]. For example, the unmarked Ramsey K10 certificate uses 356.44 checker CPU-seconds against 1,725.13 source CPU-seconds (20.66%). A 20-module sample of the retained 190-module Ramsey prefix uses 35.91% in aggregate. That prefix covers 190 of 444 targets and is

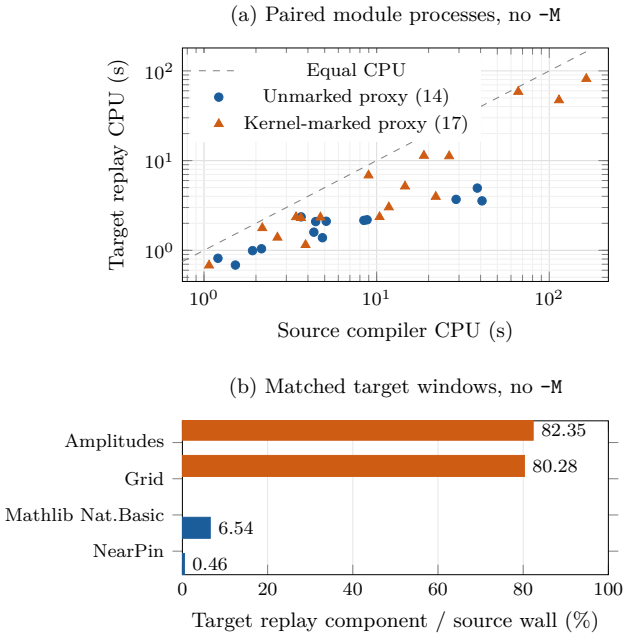


Figure 1: Two views of checking cost, all without `-M`. (a) Thirty-one complete module-process pairs; colors are source-marker proxies, whose distributions overlap. Imports are trusted in target replay. (b) Ordinary controls (blue) and computational certificates (orange), using matched target read/map/replay wall windows after dependency preparation. CPU in (a) and wall time in (b) have separate denominators. Design implication: expose the checking stage and amortize preparation; a closure average can hide an expensive certificate.

dependency-ordered; its sample describes the retained prefix.

Whole closures dilute expensive targets with cheaper dependencies. Checkpointed Falconer checking costs 2,038.44/32,292.60 CPU-seconds (6.31%); matched checkpointed Mathlib costs 91.15/1,018.22 (8.95%). Their separate wall ratios are 9.26% and 14.67%. SubexponentialSAT primary accounting reports 3,399.30 checker CPU-seconds for all 14,721 accepted modules, or 7.25% of complete source work. Using only the 3,977 new targets' source cost gives 15.79% for the same whole-closure checker numerator. Its independent reference controls remain pending. This denominator change illustrates why a single ratio cannot stand for every boundary.

3.2 Full-corpus work and its distribution

A per-module campaign reports the native compilation work actually observed; a complete library total requires complete coverage. The frozen frame has 163,097 source units; executable modules and the configuration unit retain distinct statuses. Every successful record identifies its source and measurement commits, P/E pool, native

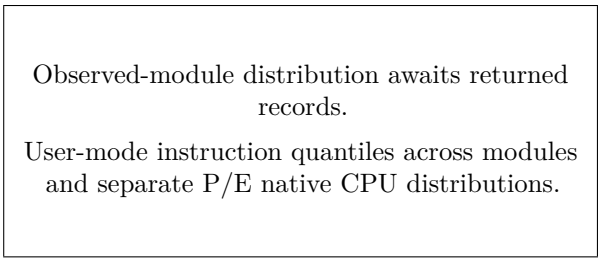


Figure 2: The observed-module distribution uses user-mode instructions, chosen after the failed total-instruction gate, and separate P/E CPU curves. Completion coverage accompanies each curve; a cutoff subset is not extrapolated. Missing costs stay unobserved and P/E seconds are not converted. Design implication: expose a heavy tail and its execution conditions before summarizing a library by one cost.

user/system CPU, wall time and user-mode instructions. Failed attempts, interruptions, blocked dependents and unattempted inputs remain separate; none is assigned zero cost. The total-instruction gate failed as described in Section 2; choosing user-mode instructions afterwards does not turn it into a pass.

The campaign is in progress; its cutoff ledger has not returned. The report will give successful native modules against the frozen frame, configuration units and the remaining failed, blocked or unattempted scope. If coverage is incomplete, Figure 2 and P/E CPU sums describe only the successfully observed subset, rather than a library total.

The original 42 probability-sample observations remain a calibration panel, with their original weights and broad nominal uncertainty. They are not pooled into the census or used to impute absent module costs. Calibration must retain each new observation's CPU pool; an E-core/P-core ratio is not a conversion factor. Machine elapsed wall time is reported only from the actual continuous run, separately from summed module-process wall intervals. Module-level joins with lexical markers and default-graph membership support cost-by-coverage analysis when those labels are supplied.

External cost comparison. The earlier tooling report builds 122,458 modules at `fd4aeeb2` on 32 cloud vCPUs in 13.8 wall-hours [10]. Its approximate 410 CPU-hours comprise roughly 330 hours of compilation and 80 hours of Comparator checking. Compilation CPU is extrapolated from a quiet 26-module laptop sample, with estimated ±30% uncertainty. That estimate is an external comparison, not an observed per-module CPU ledger on our revision. The report's 26-module kernel-skipping ablation and roughly 30,900 cloud profiles remain build-side phase precedents. Our census retains its different revision, module population, hardware and post-hoc instruction criterion; no calibrated agreement is presumed.

4 ProofServe design

ProofServe turns the cost and coverage findings into separate `check`, `audit` and compatibility-only `serve` interfaces. Its original-kernel route preserves mathematical checking rules while adding explicit coordination and artifact trust.

4.1 Check and resume a dependency closure

A checked prefix amortizes preparation while preserving each module's declared dependency context. `proofserve check <project>` discovers compiled local roots or uses explicit roots, traverses imports, and reports missing artifacts. The checker builds a graph from ordinary declarations' referenced constants. Ready declarations run against a checked frontier using the original kernel operation; the coordinator validates returned identities and commits the next frontier after all workers in a wave join. Nonordinary declarations and their ordinary prerequisites use the original replay path before the remaining graph runs. Unsupported local structure retains conservative fallback.

Two review rounds exposed false acceptance. Calling the original kernel was insufficient when the coordinator selected different artifacts or reused the wrong evidence. The first independent execution review of `2df1d9d7` found complete, exit-zero results after adding a bad-proof module part to certified inputs, and after copying an authentic certificate from another project. The no-certificate and official paths rejected the same bad inputs. Repairs bind the current project, resolved artifact family, search path, toolchain, ABI and executable identity, and publish receipts through checked writes and durable atomic replacement.

A subsequent independent code-only review of `e0c92d51` identified further input-resolution and context risks; its proposed tests had not run. M then reproduced a good checkpoint-directory artifact hiding the bad project artifact, sibling-constant borrowing and identical-definition import conflicts. A segregated-module fixture also omitted a declared axiom from a complete receipt. The `3fe83c5a` repair resolves snapshots explicitly, replays conflicting interfaces in their original context and collects axioms on every successful route. Worker events use a separate validated channel; a nonzero child exit prevents completion. Event/exit precedence was tested with a saved-result control. The proposed project-IR printing attack and Quot counterexample were not reproduced and are not counted as false-acceptance evidence.

These are defects in our coordination, not in Lean's kernel. The current native path explicitly refuses terms requiring project IR, an operational boundary that differs from official execution. Official default and fresh

also differ on the supplied stale/incomplete-part artifact. General malformed-artifact equivalence, complete retained-case module-local reference coverage and independent execution review of the latest repair remain open. The review reports and regression outcomes are retained in `packaging/checker-review-*.md` and `checker/results/e0-review-repair/`; local certificate metadata remains trusted, rather than a tamper-resistant proof.

Two prototype interventions explain these choices. Correcting our missing persistent marking reduced eight-worker CPU by 14.13% and wall time by 5.13% in a same-executable comparison. Replacing whole-module fallback with a dependency-closed nonordinary prefix reduced a paired wall median from 257.66 to 196.67 seconds (23.67%), with CPU increasing 0.106%. These results identify engineering causes in the prototype; the release ablations test their integrated effect separately. Persistent marking occurs between joined waves. The tool exposes both disabling flags for that evaluation.

Completed segments serialize their successful prefix for reuse. Toolchain, artifacts, Lake lock and checkpoint configuration must match; a directory lock protects a live invocation. An interrupted segment remains incomplete. Receipt v1 distinguishes the accepted prefix, current-invocation CPU and previously completed segment CPU, so a cache hit is identifiable. It records Lean and dependency versions, module-set hash, accepted and rejected modules, reported axiom sets and CPU/wall clocks. A restart experiment charges restoration and restart overhead.

4.2 Audit coverage and explain trust

Audit connects source inventory, the default import graph and the actual checking receipt. `proofserve audit <project>` masks comments and strings before counting kernel-decision sites and extracts imports. Supported declarative Lake configuration gives default-root coverage; executable or unsupported configuration yields an explicit unknown unless evaluated roots are supplied. Supplied roots retain their supplied status. The receipt contributes its checked compiled closure and reported axioms; a partial receipt reports the last successful prefix. These lists enumerate axiom declarations in accepted modules, rather than minimal per-theorem dependency sets. Per-module costs require explicit source-bound observations.

The tool trusts Lean's kernel plus its artifact reader, context metadata, checked-token FFI, quiescent marking and editable local checkpoints. Recorded file identities are noncryptographic; receipt reuse is not remote attestation or a proof of checkpoint integrity. Internal symbols and serialized metadata require a matching SDK. Unknown toolchains fail clearly or explicitly select the matching official checker, identifying that path and its scope. The current receipt lacks a binding to the current source digest; audit exposes this gap. Thus acceptance, source coverage and reported axiom declarations remain separate answers.

Closure	Workers	CPU / wall	Set
Mathlib Nat.Basic	Pending	Pending	Pending
Grid	Pending	Pending	Pending
Barrier	Pending	Pending	Pending

Table 1: Release-tool scaling, awaiting the complete committed matrix. Three repeats per worker count support separate CPU and wall summaries. Design implication: assess latency savings together with work inflation; prototype measurements cannot establish release performance.

5 Evaluation: performance and verification coverage

The prepared evaluation separates final performance, common-scope acceptance and audit findings; final frozen-tool results are still awaiting return. Every admitted system result must name its executable and project commits, toolchain, actual module set and complete timing boundary. Historical prototypes explain causes but cannot populate the release tables.

5.1 RQ3: original-kernel scaling and alternatives

Dependency-aware parallel replay targets elapsed time while preserving an explicit account of total CPU and additional trusted code. The release matrix checks Mathlib Nat.Basic, Grid and Barrier at 1, 2, 4 and 8 workers, with three fresh-checkpoint processes per point. Mathlib here denotes the Nat.Basic closure. At eight workers, Mathlib and Grid each receive three no-persistence and three no-split-fallback runs. Wall/CPU medians and ranges accompany acceptance parity and memory observations (Table 1).

Official default and `--fresh` comparisons require matching ledgers. Lean 4.34.1 already launches module-local tasks; its default path trusts loaded imports and replays local declarations, whereas `--fresh` replays the imported final constant map sequentially [4]. ProofServe's dependency-ordered per-module ledger is a third boundary. The evaluation reports complete command costs and common-scope acceptance; unmatched work is identified before considering speedup ratios.

An independent kernel offers another performance/trust tradeoff. The earlier report completes 416 Comparator configurations in 80.1 active hours [10]. A paired sweep finds median nanoda/Lean time 0.21, using four nanoda threads against one Lean thread, excluding export. The pinned nanoda setup,

including an enlarged checker stack, accepts 401 of 416 configurations; 15 encounter operational failures such as stack exhaustion or capacity limits, with later revisions recovering most failing cases. Nanoda is a separately implemented kernel [9]; ProofServe instead uses Lean's original kernel plus coordination. The report's result motivates this alternative, without establishing which route is faster under our evaluation conditions.

Library-checking coverage. A stopped library check reports how much was verified. The bounded attempt has not returned a final scope count. The report records unique accepts against the input frame, attempted roots, proof rejections, operational stops and unattempted scope, with tool/input commits and available official references. A verified prefix establishes that scope; unfinished work has no verdict or whole-library parity claim.

5.2 RQ4: default targets miss a third of decision sites

The default import graph misses 19.78% of source modules and 33.30% of lexical kernel-decision sites at our corpus pin. It covers 130,834 of 163,097 source modules, leaving 32,263 outside; the omitted files contain 451,008 of 1,354,379 sites. The earlier report finds 30,697 files outside its default graph, about 25% of its 122,459-node source inventory including the root [10]. The revisions and root-count conventions differ, but both demonstrate that a successful default build covers a subset of the source tree. The 33.30% figure describes decision sites, not module count.

Lexical density makes the workload difference visible even before timing. After masking comments and strings, the AI corpus contains 1,354,379 kernel-decision sites in 5,848 files; the pinned Mathlib subtree has seven sites in four of 8,529 files. Normalizing by decimal source megabytes gives a 10,668-fold density ratio. An audit should expose these labels alongside measured costs and uncertainty, rather than convert a site count into CPU work.

Repeated verdicts provide a second, separate kind of evidence. A historical panel checks 2,524 eligible stored-proof targets from three modules at 1, 2, 4 and 8 workers, three repeats each: 30,288 accepted checks and 22,716 one-worker/higher-worker comparisons. Verdicts, diagnostics and allocation counts agree in every observed pair, while elapsed time varies. These fixed-panel observations complement the installed-package rejection controls below.

The prepared external set pins Newspeak at `e2232d8f`, Solar at `f4f5c9a1`, and the public `samuelburbulla/proofs` collection at `3042904c`. The latter describes itself as AI-generated proofs; the model and human editing fraction are undisclosed. The final run must use an extracted frozen installation, fresh checkpoints and the original compiled inputs, with a common-root official comparison. Failed or resource-interrupted calls are operational outcomes rather than kernel rejection; incomplete whole-request CPU stays unknown.

Project	Final checking costs	Parity
Newspeak	Pending	Pending
Solar	Pending	Pending
AI proofs	Pending	Pending

Table 2: Final installed-tool external evaluation slots. CPU and wall seconds are separate; the artifact retains repeat counts and full ranges. Official default trusts imported proofs, so common local-root parity does not establish equal full-closure obligations or a speedup ratio. Design implication: retain scope, failures and input/tool commits.

Audit gives actionable coverage and axiom findings. Static audit covers Newspeak's 105 local files and Solar's 12 of 32 files in the default import graph. Solar's 20 omitted files are inputs to a separate EVM checking flow, which this evaluation did not execute. Newspeak's receipt lists a project `native_decide` axiom alongside declared closure axioms; a declaration inventory alone cannot assign those axioms to a particular theorem. Project pins, source-repair details and all failed outcomes are retained in `data/external-projects/`.

Historical portable-package CI on commit `adf816ae` installs a relocated archive, checks and audits a fixture, compares official default/fresh controls, and rejects loose-bound-variable, unknown-constant and wrong-sort mutations through the original kernel. The full container passes installed-project and explicit unsupported-version controls with networking disabled. These are finite historical functional qualifications; they do not certify the current repaired binary or final timing. The real interrupted/resumed versus uninterrupted cost pair still awaits the fixed matrix. The existing `serve` interface remains Kimina-compatible; service features are outside this evaluation.

Artifact and reproduction. The artifact is at https://github.com/yunwei37/proofserve. Run `proofserve check /path/to/project`, then `proofserve audit /path/to/project`; build the native entry point with `cd checker; lake build`. The evidence map retains revisions, raw outcomes and clock definitions. `docs/paper/generate_census.py` and `docs/paper/generate_system.py` generate the figures and tables from committed CSVs, with explicit partial-coverage reporting. Preview output cannot populate the manuscript.

6 Limitations and related work

Selected cost differences do not define a universal two-class law. The calibration panel has two draws per stratum and fragile heavy-tail uncertainty; lexical markers are proxies, not runtime frequencies. Selected closures overlap, module-local replay trusts imports, and target windows exclude shared preparation. Cross-protocol ratios, cache/order effects, hardware differences and provisional SubexponentialSAT controls limit generalization. Partial campaign distributions describe observed modules and may be biased by completion order; whole-library totals require full coverage. Dynamic phase weights remain unmeasured.

Finite verdict agreement does not prove arbitrary-extension safety, future budget sufficiency or soundness of the coordinator. Reported axiom unions retain their actual scope: acceptance can depend on axioms, unsafe/partial exclusions remain visible, and semantic statement faithfulness requires separate review. Current source binding, malicious checkpoint integrity, executable Lake configuration and uncompiled/generated sources require explicit additional evidence. Release scaling and resumption costs remain incomplete. Historical external observations have one successful arm each and common-root rather than full-dependency official parity; final evaluation is pending; the AI disclosure is a collection-level attribution, not an independent authorship audit.

Parallel proof checking and asynchronous tasks have precedents in Isabelle, Coq and Lean [13, 1, 3]. Independent checkers and Lake's shipped workflow already address rechecking [6, 7]; the same-corpus tooling report also proposes standardized timing, axiom and hash receipts [10]. Kimina and AXLE supply verification services [12, 14]. Our contribution is the measured boundary between checking and recomputation and its integration into a usable original-kernel tool, rather than new parallelism, receipt or serving mechanisms.

7 Conclusion

Matched measurements show both cheap proof checking and substantial certificate recomputation. The per-module campaign will report its observed coverage and costs, retaining failed-gate and pool boundaries even if it stops short. Default coverage and repeatable verdicts answer different verification questions. ProofServe makes these boundaries explicit through check, audit and scoped receipts. Review-driven repairs strengthen the tested input and receipt boundaries; independent review and the pending final evaluation must establish the release's remaining correctness and performance claims.

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[3] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[4] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[5] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[6] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[7] Lean FRO contributors. Lean 4.35.0 release notes: Lake checking and comparator, 2026.

[8] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[9] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[10] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[11] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[12] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[13] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[14] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.